UNIVERSIDAD DEL VALLE • INGENIERÍA DE SISTEMAS

CLI SEGURA: PROTECCIÓN DE LOGS EN REPOSO

CRIPTOGRAFÍA SIMÉTRICA, HASHING FORENSE Y AUDITORÍA ATÓMICA

Taller 2 de Criptografía y Seguridad en Redes • Escuela de Ingeniería de Sistemas y Computación • Santiago de Cali, 2026.

ESTÁNDAR NATIVO

100%

LIBRERÍA ESTÁNDAR DE GO

GO 1.22+ • RFC 8785 • ZERO CGO • NIST FIPS 180-4

EJECUCIÓN DEL PIPELINE

COMANDO DE ENTRADA CLI

```
$ go run ./cmd/cli --auto -n 25
[01] Lote: 25 eventos configurados
[03] Hashing: SHA-256 canónico RFC 8785 (JCS)
[06] Cifrado: AES-256-GCM + GMAC Tag (AEAD)
[10] Auditoría: Integridad 100% preservada
```

REPRODUCIBLE: GO RUN ./CMD/CLI --AUTO -N <N>

EQUIPO DE TRABAJO & DISTRIBUCIÓN DE RESPONSABILIDADES

JOSE ANTONIO CASTELBLANCO
INTEGRANTE 1
Modelado de Dominio & Generador Estocástico

BRAULIO ROBLEDO
INTEGRANTE 2
Integridad, Hashing SHA-256 & Auditoría Dual

STEVEN NIEVES
INTEGRANTE 3
Cifrado AES-256-GCM & Gestión de Secretos

JUAN JOSÉ BOLAÑOS
INTEGRANTE 4
Orquestación Pipeline, CLI & Tampering Tests

CONTEXTO OPERATIVO • SISTEMA DE GESTIÓN ACADÉMICA

VECTOR DE AMENAZA: MANIPULACIÓN EN REPOSO (DATA AT REST)

IMPACTO DE NEGOCIO

CALIFICACIONES CRÍTICAS

- Alteración de notas compromete actas oficiales, becas y títulos.
- Fraude quirúrgico: variar una décima (4.4 a 4.5) es indetectable en texto claro.
- Riesgo severo ante intrusos locales o personal con privilegios de root.

CRÍTICO

SEVERIDAD ANTE FRAUDE ACADÉMICO

SUPERFICIE DE ATAQUE

EXPOSICIÓN EN CLARO

```
# Adversario lee o altera notas en disco
$ cat output/academic_log.json | grep "grade"
  "grade": 4.5
$ sed -i 's/"grade": 4.5/"grade": 5.0/' \
  output/academic_log.json
```

- **Sin Cifrado:** Exfiltración masiva de datos en claro.
- **Sin Resúmenes:** Modificación inadvertida sin traza forense.

```
VULNERABILIDAD:
academic_log.json [Vulnerable a Manipulación]
```

MITIGACIÓN PROPUESTA

ARQUITECTURA DEFENSIVA

- **Blindaje AES-256-GCM:** Cero datos en claro almacenados en reposo.
- **Integridad de Doble Nivel:** Detección macro (archivo) y micro (registro).
- **Carpeta 0700:** `drwx------` (Directorio: lectura, escritura y acceso solo dueño).
- **Archivos 0600:** `-rw-------` (lectura y escritura únicamente al usuario propietario del archivo).

DESGLOSE: D=DIR | R=LECTURA(4) | W=ESCRITURA(2) | X=ACCESO(1)

```
$ stat -c "%a %A %n" output/ output/*.enc
700 drwx------ output/ (rwx dueño, sin grupo/otros)
600 -rw------- academic_log.enc (rw dueño, sin grupo/otros)
```

ESTÁNDAR ISO/IEC 27001 • MODELO STRIDE

ATRIBUTOS DE CALIDAD & MODELO DE MITIGACIÓN

MATRIZ OPERATIVA DE SEGURIDAD

ATRIBUTO	MECANISMO EN GO	AMENAZA MITIGADA
CONFIDENCIALIDAD	AES-256-GCM (cifrado AEAD en reposo)	Exfiltración no autorizada de calificaciones
INTEGRIDAD GRANULAR	SHA-256 sobre sub-payload	Mutación quirúrgica de notas o UIDs
INTEGRIDAD GLOBAL	SHA-256 Streaming O(1)	Inserción, borrado o reorden de eventos
AUTENTICIDAD	Tag GMAC (firma de autenticación 128-bit)	Inyección de ciphertext forjado o bit-flipping
INMUNIDAD TIMING	`subtle.ConstantTimeCompare` (tiempo constante)	Ataques de canal lateral por microsegundos

TESTS AUTOMATIZADOS DE VULNERABILIDAD:

```
$ go test -v -run "TestCrypto|TestAudit" ./...
PASS: TestAESGCM_BitFlippingDetection (0.01s)
PASS: TestConstantTimeCompare_TimingInvariant (0.00s)
```

POLÍTICAS DE SISTEMA

PRINCIPIOS CLAVE

1. MÍNIMO PRIVILEGIO (OS)
Directorio `output/` con permisos `0700` y secretos en `0600` (lectura y escritura únicamente al usuario propietario del archivo).

2. SEPARACIÓN DE SECRETOS
Ciphertext (`.enc`) desacoplado físicamente de llaves y nonces.

3. TIEMPO CONSTANTE
Cero fugas de información temporal en comparaciones forenses.

DEFENSA EN PROFUNDIDAD: NIST & ISO 27001

DISEÑO DE DATOS & RFC 8785 (JSON CANONICALIZATION)

ESTRUCTURA DEL LOG: ACADEMIC_LOG.JSON

ESPECIFICACIÓN FORMAL

REGLAS DE DOMINIO

- **Identificadores Opacos:** USR-STU-*, USR-DOC-*, SUB-*.
- **Distribución Estocástica:** Uniforme U[0.0, 5.0] con math/rand/v2 (PRNG de dominio, no criptográfico).
- **Trazabilidad de Deltas:** previous_grade, new_grade y reason.

INSPECCIÓN VÍA CLI / JQ:

```
$ jq '.[0] | {event_id, payload_hash}' \
  output/academic_log.json
{
  "event_id": "EVT-REG-00001",
  "payload_hash": "a591a6d4..."
}
```

EVENTO 1: REGISTRO_NOTA

ANATOMÍA DEL SELLADO

```
{
  "event_id": "EVT-REG-00001",
  "event_type": "REGISTRO_NOTA",
  "payload": {
    "student_uid": "USR-STU-001",
    "grade": 4.5,
    "evaluation": "Parcial 1",
    "timestamp": "2026-10-29T12:00:00Z"
  },
  "payload_hash": "a591a6d4...9ad9f146e"
}
```

El hash sella **exclusivamente** el sub-objeto payload antes de escribir el contenedor.

EVENTO 2: MODIFICACION_NOTA

DELTA AUDITABLE

```
{
  "event_id": "EVT-MOD-00002",
  "event_type": "MODIFICACION_NOTA",
  "payload": {
    "student_uid": "USR-STU-001",
    "previous_grade": 4.5,
    "new_grade": 5.0,
    "reason": "Recalificación"
  },
  "payload_hash": "3f42b89c...14a2b901f"
}
```

Historial inmutable: auditoría de notas previas sin permitir borrado destructivo del historial.

PASO 3 • DISEÑO CRIPTOGRÁFICO FORMAL

RESOLUCIÓN DE CIRCULARIDAD EN HASHING DE PAYLOAD

PARADOJA DE AUTORREFERENCIA

EL PROBLEMA MATEMÁTICO

Si el hash se calcula sobre todo el objeto E y se almacena en su interior, los bytes mutan inmediatamente:

$$H(E \cup \{h\}) \neq h$$

- Autosellar una entidad con su propio digest genera recursión infinita.
- Cada inyección del hash invalida irreversiblemente el hash anterior.
- Solución: Aislamiento estricto de contenedor y payload inmutable.

TEST DE NO-CIRCULARIDAD:

```
$ go test -v -run TestCanonicalPayloadHash ./...
PASS: TestCanonicalPayloadHash_NoCircularity (0.00s)
```

ARQUITECTURA DE AISLAMIENTO EN 4 PASOS

PIPELINE DE SELLADO DE TRANSACCIÓN

1. AISLAMIENTO ESTRUCTURAL
Bifurcación en contenedor exterior y sub-objeto inmutable `payload`.

2. CANONICIDAD RFC 8785 (JCS)
Orden lexicográfico de claves y sin espacios para garantizar hashes deterministas idénticos.

3. CÁLCULO SHA-256
El digest se computa **únicamente** sobre los bytes de `payload`.

4. INYECCIÓN EXTERNA
El digest se escribe en `payload_hash`, dejando el payload 100% intacto.

MODELO FORMAL DE HASHING DE TRANSACCIÓN

$$\text{PAYLOADHASH} = \text{SHA256}(\text{CANONICALJSON}(\text{EVENT.PAYLOAD}))$$

NIST FIPS 180-4 • ARQUITECTURA DE ALTO RENDIMIENTO

SHA-256 NATIVO Y PROCESAMIENTO POR STREAMING O(1)

ROBUSTEZ NIST FIPS 180-4

ESPACIO CRIPTOGRÁFICO

- **Colisiones:** Margen de 2^{128} ops (imposible hallar dos mensajes con igual hash).
- **Preimagen:** 2^{256} ops (imposible revertir el hash al mensaje original).
- **Aceleración Hardware:** Instrucciones Intel SHA-NI y ARMv8 automáticas en Go.

2^{256}

ESTADOS POSIBLES DEL RESUMEN

PREVENCIÓN DE DOS / OOM

STREAMING O(1) (MEMORIA CONSTANTE)

```
// io.Copy transmite por buffers de 32 KB
hasher := sha256.New()
if _, err := io.Copy(hasher, file); err != nil {
    return "", err
}
digest := hex.EncodeToString(hasher.Sum(nil))
```

- Cero precarga de archivos gigantescos en RAM.
- Inmunidad al OOM Killer (evita que el SO liquide el proceso).
- Consumo de memoria constante `O(1)` plano (buffer 32 KB).

COMPLEJIDAD ESPACIAL: O(1) CONSTANTE

INTEROPERABILIDAD FORENSE

GNU COREUTILS

Los testigos se persisten con formato estándar POSIX: `<hash_64_hex> <archivo>`

Verificación pericial universal en terminal:

```
$ sha256sum -c output/academic_log.json.sha256
output/academic_log.json: OK

$ sha256sum -c output/academic_log.enc.sha256
output/academic_log.enc: OK
```

VALIDACIÓN INDEPENDIENTE SIN DEPENDER DE GO

NIST SP 800-38D & NIST SP 800-57

CIFRADO AUTENTICADO EN REPOSO: AES-256-GCM

MODO AEAD (CIFRADO AUTENTICADO: PRIVACIDAD + INTEGRIDAD)

GALOIS/COUNTER MODE (GCM)

- **Confidencialidad Simétrica:** Cifrado en flujo basado en AES-CTR con llave de 256 bits.
- **Autenticidad e Integridad:** Tag de autenticación Galois (**GMAC**, firma de 128 bits).
- **Mitigación de Bit-Flipping:** Alterar 1 bit en ciphertext corrompe el tag GMAC y aborta descifrado.

DESGLOSE: R=LECTURA(4), W=ESCRITURA(2), X=EJECUCIÓN(1):

```
$ stat -c "%a %A %n" output/academic_log.enc output/crypto_metadata.json
600 -rw------- academic_log.enc (r+w dueño, cero resto)
600 -rw------- crypto_metadata.json (r+w dueño, cero resto)
```

GESTIÓN DEL CICLO DE VIDA DE SECRETOS

POLÍTICAS CRIPTOGRÁFICAS NIST

CSPRNG (CRYPTO/RAND)
Entropía de `getrandom(2)` del kernel (impredecible).

CLAVE DE 256 BITS
32 bytes de entropía pura (2^{256} combinaciones).

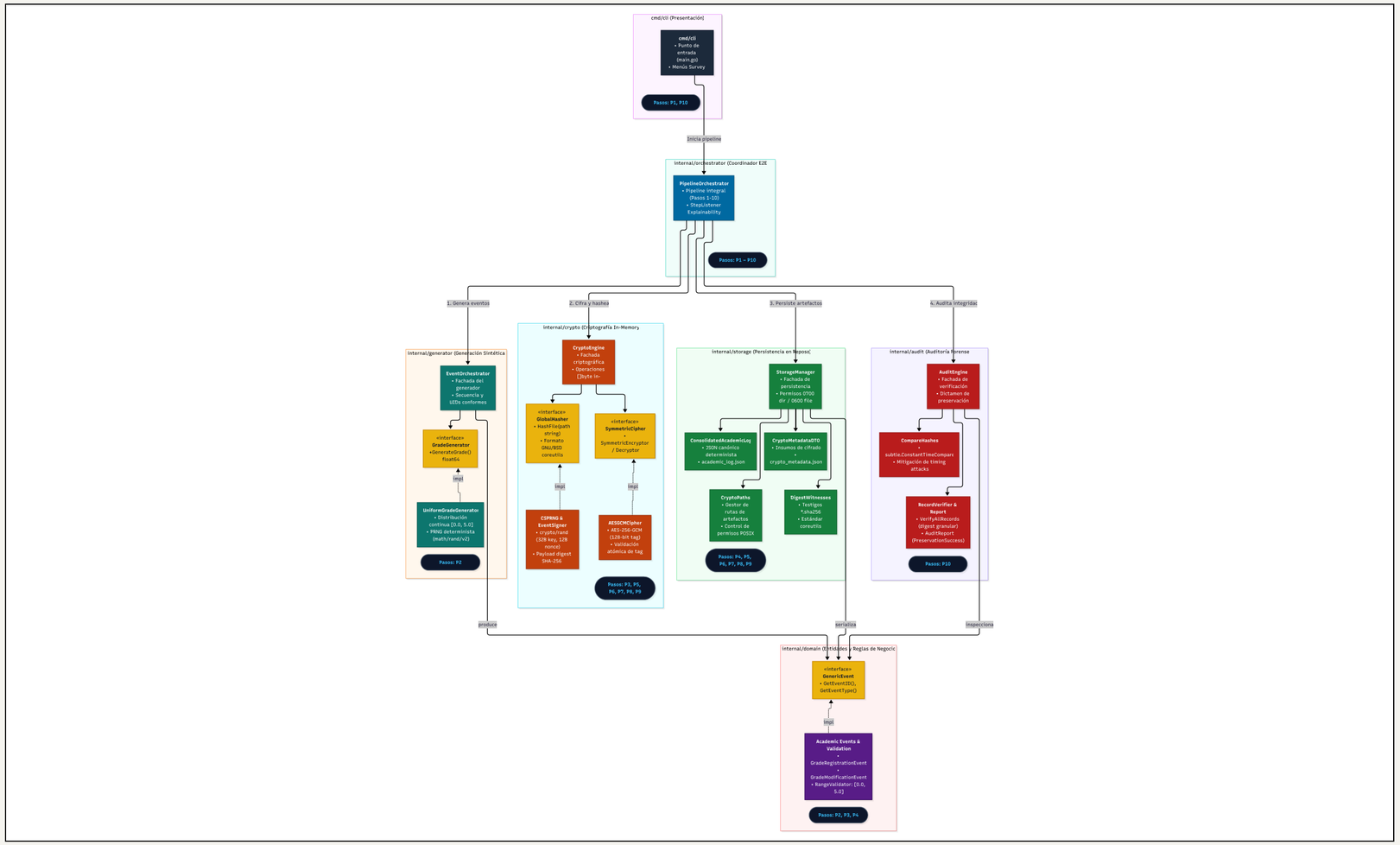
NONCE ÚNICO DE 96 BITS (IV)
Vector de 12 bytes aleatorios frescos por corrida (cero reuso).

PERMISOS POSIX 0600 (-RW-------)
0600 (lectura y escritura únicamente al usuario propietario del archivo).

SEPARACIÓN FÍSICA DE PRIVILEGIOS
Tener el archivo cifrado `academic_log.enc` no compromete el log sin las llaves aisladas en `crypto_metadata.json`.

ARTEFACTO DE DISEÑO 1 • MODELO DE COMPONENTES

ARQUITECTURA MODULAR Y DESACOPLAMIENTO DE PAQUETES



PRINCIPIOS DE ARQUITECTURA

- **Fachada (Facade):** `PipelineOrchestrator` unifica generador, criptografía, almacenamiento y auditoría.
- **Inversión de Dependencias:** Interfaces abstractas desacopladas en `domain`.
- **Grafo Acíclico (DAG):** Cero dependencias circulares y cero CGO.

100% GO STANDARD LIBRARY

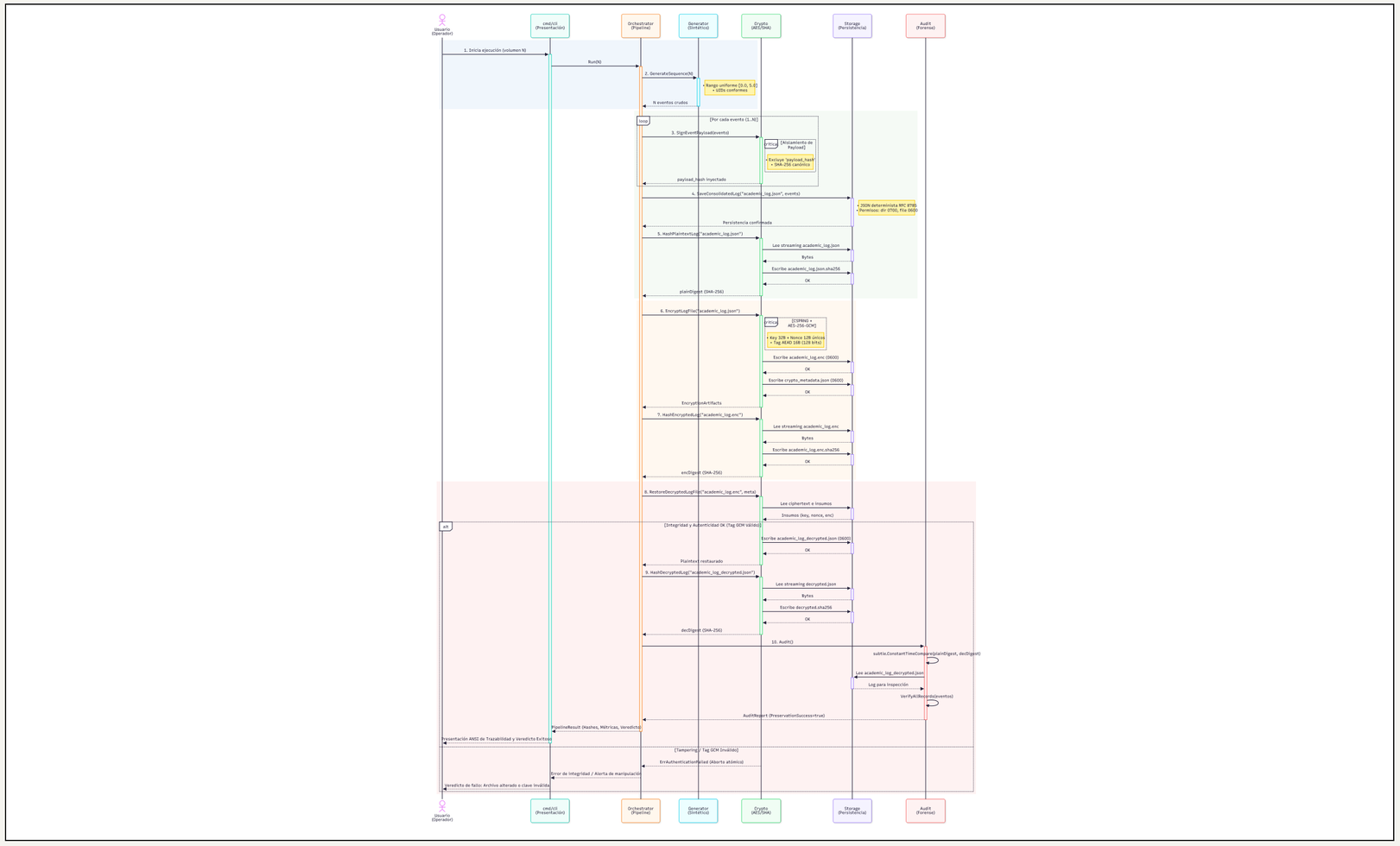
PERSISTENCIA EN OUTPUT/

```
$ ls -1 output/
academic_log.json          # Claro
academic_log.json.sha256   # Testigo P5
academic_log.enc           # AES-GCM
crypto_metadata.json       # Key/Nonce
academic_log_decrypted.json # Decrypt
*.sha256                   # Coreutils
```

ARTEFACTOS DESACOPLADOS FÍSICAMENTE

ARTEFACTO DE DISEÑO 2 • UML 2.5 INTERACTION DIAGRAM (VECTORIAL NATIVO)

COMPORTAMIENTO SECUENCIAL: PASOS 1 AL 10



FASE 1: INGESTIÓN Y SELLADO

PASOS 1 AL 5

- **[01-02]** Volumen N y notas estocásticas U[0.0, 5.0].
- **[03]** Hashing canónico RFC 8785 (anti-circularidad).
- **[04-05]** Log plano y digest global SHA-256.

SELLADO ATÓMICO PRE-ALMACENAMIENTO

FASE 2: BLINDAJE Y AUDITORÍA

PASOS 6 AL 10

- **[06-07]** Cifrado AES-256-GCM + GMAC Tag y hash.
- **[08-09]** Descifrado autenticado y hash de archivo restaurado.
- **[10]** Auditoría dual con `subtle.ConstantTimeCompare`.

```
$ go run ./cmd/cli --auto -n 50
```

PASO 10 • VERIFICACIÓN FORENSE DUAL

AUDITORÍA FORENSE DUAL & CONSTANT-TIME COMPARE

DOBLE NIVEL DE INTEGRIDAD

VERIFICACIÓN MACRO Y MICRO

- **Macro (Global):** Compara `academic_log.json.sha256` con `academic_log_decrypted.sha256` (integridad bit a bit del archivo).
- **Micro (Granular):** `VerifyAllRecords()` recalcula el hash de cada payload individual contra su `payload_hash`.
- **Aislamiento Quirúrgico:** Identifica con exactitud el UID del alumno y el ID del evento vulnerado.

EJECUCIÓN DE AUDITORÍA FORENSE:

```
$ go run ./cmd/cli  # Opción 6: Auditoría
[+] Integridad Global (SHA-256): 100% MATCH
[+] Integridad Granular: 25/25 registros VÁLIDOS
```

CANALES LATERALES (SIDE-CHANNEL ATTACKS)

MITIGACIÓN DE TIMING ATTACKS

Operadores estándar (==) retornan en el primer byte desigual (tiempo variable). `subtle.ConstantTimeCompare` evalúa todos los bytes en tiempo constante.

```
// Comparación en tiempo constante invariante O(N)
isEqual := subtle.ConstantTimeCompare(
    []byte(origHash),
    []byte(decHash),
) == 1
```

- Tiempo de CPU idéntico independientemente de bytes coincidentes.
- Cero fuga de microsegundos que permita deducir hashes.

GARANTÍA MATEMÁTICA CRIPTOGRÁFICA
Inmunidad total frente a ataques de temporización en red o local.

EVIDENCIA EXPERIMENTAL • EJECUCIÓN DEL PIPELINE

RESULTADOS EN CONSOLA & TRAZABILIDAD DE HASHES

EJECUCIÓN DEL PIPELINE

TERMINAL RUNNER

```
$ go run ./cmd/cli --auto -n 25
--- 🔄 TRAZABILIDAD DE PASOS (1 AL 10) ---
[01] Volumen: 25 eventos configurados
[02] Generación: 23 Reg / 2 Mod U[0.0-5.0]
[03] Hash Payload: SHA-256 canónico RFC 8785
[04] Consolidación: output/academic_log.json
[05] Hash Claro: academic_log.json.sha256
[06] Cifrado Reposo: AES-256-GCM + GMAC OK
[07] Hash Cifrado: academic_log.enc.sha256
[08] Descifrado: Tag GMAC 128-bit verificado
[09] Hash Descifrado: academic_log_decrypted.sha256
[10] Auditoría: 25 válidos (0 corruptos)
```

COMANDO: GO RUN ./CMD/CLI --AUTO -N 25

CADENA FORENSE DE CUSTODIA (SHA-256)

RESÚMENES EN LAS TRES ETAPAS CLAVE

ETAPA DE PIPELINE	RESUMEN CRIPTOGRÁFICO SHA-256 (DIGEST HEX)
1. Log Original (P5) ACADEMIC_LOG.JSON	416d902757ac1321e1af...1cd0f38d35bd6c66
2. Log Cifrado (P7) ACADEMIC_LOG.ENC	a4762936283f3d1a5cad...ba31afb091e61218
3. Log Descifrado (P9) ACADEMIC_LOG_DECRYPTED.JSON	416d902757ac1321e1af...1cd0f38d35bd6c66

```
[ ✔ ] VERIFICACIÓN EXITOSA: INTEGRIDAD Y CONFIDENCIALIDAD PRESERVADAS
• Confidencialidad: Cifrado con AES-256-GCM nativo.
• Integridad Global: Hash Original == Hash Descifrado (100% Match).
• Integridad Granular: Payloads individuales 100% válidos.
• Conclusión: Preservación fiel sin pérdida ni alteración de datos.
```

RESISTENCIA FORENSE ACTIVA

PRUEBAS DE ROBUSTEZ Y DETECCIÓN DE TAMPERING

ESCENARIO 1: CORRUPCIÓN EN REPOSO

BIT-FLIP EN ACADEMIC_LOG.ENC

Se altera 1 byte en el archivo cifrado. AES-GCM verifica el Tag GMAC antes de escribir:

```
# Mutación de 1 byte en offset de ciphertext
$ python3 -c 'b=bytearray(open("output/academic_log.enc","rb").read()); b[0x4A]^=0xFF; open("output/academic_log.enc","wb").write(b)'
$ go run ./cmd/cli --auto -n 5
[06] Cifrado Reposo: AES-256-GCM OK
[08] Descifrado:
panic: cipher: message authentication failed
```

DEFENSA ATÓMICA AEAD

El Tag GMAC aborta el descifrado antes de exponer datos corruptos en memoria o disco.

ESCENARIO 2: ADULTERACIÓN EN CLARO

MANIPULACIÓN SELECTIVA DE NOTA

Se modifica la calificación directamente en el JSON descifrado:

```
# Alterar nota en archivo JSON descifrado
$ sed -i 's/"grade": [0-9.]*/"grade": 0.0/' \
  output/academic_log_decrypted.json
$ go run ./cmd/cli  # Opción 6: Auditoría
[!] Hashes globales SHA-256 no coinciden [MISMATCH]
[✘] EVT-REG-00003: Payload Hash Discrepancy!
```

```
[ ✘ ] ALERTA DE SEGURIDAD: VIOLACIÓN DE INTEGRIDAD
• Integridad Global: DISCREPANCIA (Hashes divergentes).
• Integridad Granular: Registro EVT-REG-00003 aislado.
• Salida POSIX: Exit Code 1 (Fallo / Alerta pericial).
```

POLÍTICA INSTITUCIONAL & ÉTICA ACADÉMICA

DECLARACIÓN DE AUTORÍA Y USO RESPONSABLE DE IA

METODOLOGÍA Y ÁREAS DE ASISTENCIA

PLANIFICACIÓN, MODELADO Y CÓDIGO

Herramientas: Google Antigravity (Gemini 3.8 Flash/Argon 4) • Anthropic Claude Code.

Se realizaron las diapositivas con ayuda de la skill https://github.com/nugrahalabib/AgentBuff-Presentation-Skills

- **1. Gestión & ClickUp:** Descomposición de requerimientos de seguridad en historias atómicas (US-1 a US-11), criterios de aceptación y sprints.
- **2. Arquitectura CLI & Streaming:** Parseo de parámetros (`--auto`, `-n`) con biblioteca estándar de Go y lectura por bloques mediante `io.Copy` para procesar archivos con memoria constante `O(1)` sin saturar la RAM.
- **3. Criptografía Nativa:** Co-diseño de AES-256-GCM (cifrado AEAD autenticado), canonización RFC 8785 (JCS determinista) y desacoplamiento para resolver la circularidad del hash.

SUPERVISIÓN HUMANA CONTINUA • ZERO CGO (100% GO NATIVO SIN C)

GOBERNANZA, AUDITORÍA Y RESPONSABILIDAD ÉTICA

VALIDACIÓN RIGUROSA (100% HUMANA)

- **Code Review Manual en PRs:** Revisión por pares obligatoria antes de integrar cambios a `main`, descartando librerías externas y blindando contra canales laterales con `subtle.ConstantTimeCompare`.
- **Verificación Empírica & Tests:** Suite exhaustiva de table-driven tests (concurrencia con `-race`, bit-flipping, corrupción de tags GMAC y permisos POSIX `0600 (lectura y escritura únicamente al usuario propietario del archivo)`).
- **Autoría Total & Comprensión:** El equipo comprende la arquitectura interna, el flujo de memoria y la criptografía aplicada, asumiendo total autoría ética y técnica del software entregado.

```
$ go test -v -race ./...
PASS: 30+ tests unitarios, concurrencia y tampering (100% OK, Cobertura 100%)
```

EVALUACIÓN FINAL & LECCIONES APRENDIDAS

CONCLUSIONES TÉCNICAS Y PROYECCIÓN A PRODUCCIÓN

LECCIONES DE SEGURIDAD

DEFENSA EN PROFUNDIDAD

- **AEAD + Hashing Granular:** AES-256-GCM da confidencialidad; el tag GMAC (autenticación) y el hash de payload evitan manipulación en reposo.
- **Rendimiento Nativo:** Cero dependencias externas y complejidad `O(1)` en memoria RAM eliminan riesgos de cadena de suministro y DoS.
- **Higiene Anti-Side-Channel:** Uso indispensable de `subtle.ConstantTimeCompare` (tiempo constante) contra ataques temporales.

CUMPLIMIENTO: 100% DE REQUISITOS DEL TALLER 2

HOJA DE RUTA A PRODUCCIÓN

PRÓXIMOS PASOS DE INGENIERÍA

1. CUSTODIA CENTRALIZADA DE LLAVES (KMS)
Migrar `crypto_metadata.json` a HashiCorp Vault o AWS KMS.

2. NO REPUDIO ASIMÉTRICO (ED25519)
Firma digital por docente con pares de llaves pública/privada.

3. INMUTABILIDAD DISTRIBUIDA (MERKLE LOGS)
Anclaje de raíces de Merkle en un append-only log auditable.

```
$ rm -rf output/* && go run ./cmd/cli --auto -n 50
```